AI Usage Log and Reflection Report

Smart Task & Productivity Management System (TaskFlow)

Course: SE ZG503 — Full Stack Application Development
Approach: Option A — Built from scratch with AI assistance
AI Tool Used: Claude (Anthropic) — claude.ai

1. AI Tools Used

Claude (Anthropic) was the primary AI tool used throughout this project. Interactions were through the chat interface at claude.ai. No IDE plugins (Copilot, Cursor) were used. Claude was consulted at specific stages for code generation, debugging help, and documentation generation — detailed in the log below.

2. AI Usage Log — Prompts & Outcomes

The table below records the key areas where AI was consulted, the nature of the prompt, what the AI produced, and what was done manually or adjusted afterwards.

Area	Prompt Given to AI (summary)	AI Output	Student Action
Project Setup	"Set up a Spring Boot 3 project with JWT auth, JPA, PostgreSQL and Lombok for a task management system"	Generated pom.xml, application.properties, base package structure	Chose problem statement; verified dependency versions matched Spring Boot 3.3.5
Backend Entities	"Create User, Task, Category entities with proper JPA relationships and enums for Role, TaskStatus, Priority"	Generated entity classes, enum files, foreign key mappings	Reviewed relationships; corrected MANAGER/EMPLOYEE business rules manually
Security / JWT	"Implement Spring Security 6 JWT filter chain — stateless, no session"	Generated SecurityConfig, JwtUtil, JwtAuthFilter, UserDetailsServiceImpl	Tested token expiry; fixed filter ordering issue manually
REST Controllers	"Write CRUD controllers for Task, Category, User with role-based @PreAuthorize"	Generated 6 controllers with standard CRUD endpoints	Added business rule: employees can only see own tasks; managers see all
Analytics & Insights	"Build an analytics service that returns task counts by status/priority and a productivity	Generated AnalyticsService and InsightService	Adjusted score formula and insight

Area	Prompt Given to AI (summary)	AI Output	Student Action
	score"	with scoring logic	thresholds to match realistic use cases
React Frontend	"Create a React + Vite + Tailwind frontend with login, dashboard, task management pages and Chart.js analytics"	Generated page components, routing, Axios API service, auth hooks	Redesigned layout; fixed chart rendering bug in AnalyticsChart.jsx (SVG arc maths were wrong)
Chart Debug	"The productivity score gauge needle is not pointing to the correct position on the arc"	Explained the SVG arc geometry issue	Manually corrected arc centre (74,72) and radius (58) by reading the SVG path values
API Docs	"Generate a complete OpenAPI 3.0 swagger.yaml for these 16 endpoints"	Generated full swagger.yaml with schemas and examples	Reviewed all response schemas; corrected InsightDTO fields
Registration Fix	"Remove role picker from Register page — all new users should default to Employee"	Updated Register.jsx removing role selector UI	Verified backend still accepts role field; tested new user flow manually

3. Parts Generated by AI vs. Manually Written

AI Generated / AI Assisted	Written / Decided by Student
• Spring Boot project scaffold & pom.xml • Entity and DTO class boilerplate • JWT security filter chain skeleton • Basic CRUD endpoint structure • React component scaffolding • Swagger YAML documentation	• Problem statement & domain choice • Role-based business rules (who sees what) • SVG arc fix for productivity gauge • Employee-only self-registration decision • Debugging CORS, JWT filter, chart bugs • Testing all flows end-to-end

4. Reflection

4.1 Did AI Help or Hinder Understanding?

Overall, AI accelerated development significantly — especially for repetitive boilerplate like entity classes, DTO mapping, and standard CRUD endpoints. However, AI was not a replacement for understanding. When the productivity gauge needle rendered at the wrong position on the SVG arc, Claude could explain the geometry but the actual debugging — reading the path coordinates, calculating centre and radius, and correcting the trigonometry — required manual analysis. Blind acceptance of AI output would have left a broken UI.

AI output also needed consistent domain-level review. For instance, the initial InsightService thresholds were generic; adjusting them to realistic productivity benchmarks required domain judgment that AI could not provide without explicit direction.

4.2 Issues Encountered Integrating AI Output

- Spring Security 6 changed the configuration API significantly from earlier versions. The AI initially generated a deprecated WebSecurityConfigurerAdapter pattern that needed to be updated to the SecurityFilterChain lambda-based approach.
- The AnalyticsChart component generated by AI used canvas gradient objects directly inside Chart.js data arrays, which broke in react-chartjs-2 v5. This required moving gradient creation into a useEffect hook with a chart reference.
- On Windows, the project folder contains an ampersand (&) in its name, which caused the npm run dev command to fail in CMD. The workaround (running Vite directly via node node_modules/vite/bin/vite.js) had to be identified manually.
- AI-generated Swagger schemas initially missed several response fields from AnalyticsDTO and InsightDTO. These were cross-checked against the actual Java classes and corrected.

4.3 What I Learned from Debugging AI-Generated Code

- Reading framework documentation is essential — AI suggestions are based on training data that may not reflect the latest API changes (e.g., Spring Security 6, react-chartjs-2 v5).
- AI generates plausible-looking code quickly, but correctness for domain-specific logic (business rules, UI geometry, role access policies) must always be verified by the developer.
- Prompt specificity matters greatly. Vague prompts like "add authentication" produced generic output; specific prompts like "implement a stateless JWT filter for Spring Security 6 that extracts Bearer tokens and sets the SecurityContext" produced directly usable code.
- AI is most effective as a pair programmer — it handles the repetitive 80%, while the developer handles the context-specific 20% that requires actual understanding.

5. Summary

Dimension	Summary
AI Tool	Claude (Anthropic), claude.ai
Approach	Option A — Build from scratch with AI assistance
Estimated AI contribution	~55% (boilerplate, structure, docs) | ~45% student (design, debugging, testing, fixes)
Biggest AI benefit	Speed of scaffolding — backend and frontend skeletons that would have taken days were produced in hours
Biggest AI limitation	Cannot reason about runtime context — chart rendering bugs, OS-level path issues, and version-specific API changes required manual debugging

Dimension	Summary
Key learning	AI is a productivity multiplier, not a replacement for understanding. Every AI-generated module was read, tested, and often adjusted before use.